

```
[USER] ls
```

```
DESCRIPTION : Liste le contenu d'un répertoire
POURQUOI    : Commande la plus utilisée de Linux. Permet de voir quels fichiers et
              dossiers existent, leurs permissions, tailles, dates de modification.
CONTEXTE    : Navigation quotidienne, vérification de présence de fichiers, scripts
              d'inventaire, diagnostic de permissions.
OPTIONS     :
  -l          Format long : permissions, propriétaire, taille, date
  -a          Affiche les fichiers cachés (commençant par .)
  -h          Tailles lisibles (Ko, Mo, Go) – combiner avec -l
  -R          Récursif : liste tous les sous-répertoires
  -t          Trie par date de modification (plus récent d'abord)
  -S          Trie par taille décroissante
  -r          Inverse l'ordre de tri
  -i          Affiche le numéro d'inode de chaque fichier
  --color=auto Colorise la sortie selon le type de fichier
  -l          Un fichier par ligne (pratique pour les scripts)
  -d          Affiche le répertoire lui-même, pas son contenu
  -F          Ajoute un indicateur : / pour dossier, * pour exécutable
EXEMPLES    :
$ ls -lah /etc
$ ls -lt /var/log
$ ls -R /home/user
$ ls -lS /usr/bin
```

```
[USER] cd
```

```
DESCRIPTION : Change le répertoire courant (Change Directory)
POURQUOI    : Se déplacer dans l'arborescence du système de fichiers.
CONTEXTE    : Navigation permanente dans le terminal, scripts shell.
OPTIONS     :
  cd ~          Va dans le répertoire home de l'utilisateur courant
  cd ..         Remonte d'un niveau dans l'arborescence
  cd -          Retourne dans le répertoire précédent
  cd /          Va à la racine du système de fichiers
  cd /chemin/absolu Navigation par chemin absolu depuis /
  cd chemin/relatif Navigation relative au dossier courant
EXEMPLES    :
$ cd /etc/nginx
$ cd ..
$ cd ~/Documents
$ cd -
```

```
[USER] pwd
```

```
DESCRIPTION : Affiche le chemin absolu du répertoire courant (Print Working Directory)
POURQUOI    : Savoir exactement où on se trouve dans l'arborescence – essentiel dans
              les scripts pour construire des chemins dynamiques.
CONTEXTE    : Diagnostic, scripts shell, orientation dans le système.
OPTIONS     :
  -L          Suit les liens symboliques (comportement par défaut)
  -P          Affiche le chemin physique réel (résout les liens)
EXEMPLES    :
$ pwd
$ pwd -P
```

```
[USER] cp
```

```
DESCRIPTION : Copie des fichiers ou répertoires
POURQUOI    : Dupliquer des fichiers pour sauvegarde, tests ou déploiement sans
              toucher à l'original.
CONTEXTE    : Sauvegardes manuelles, copie de configurations, déploiement d'apps.
OPTIONS     :
  -r / -R      Copie récursive – indispensable pour les dossiers
  -p          Préserve les attributs : permissions, timestamps, proprio
```

```

-i          Mode interactif : demande confirmation avant écrasement
-u          Copie seulement si la source est plus récente
-v          Mode verbeux : affiche chaque fichier copié
-l          Crée des liens physiques au lieu de copier
-s          Crée des liens symboliques au lieu de copier
-a          Mode archive : équivalent à -dR --preserve=all
-n          Ne pas écraser les fichiers existants
--backup    Crée une sauvegarde des fichiers écrasés

```

EXEMPLES :

```

$ cp fichier.txt /tmp/
$ cp -rp /home/user /backup/
$ cp -iv source.conf dest.conf
$ cp -a /var/www /backup/www_$(date +%F)

```

[USER] mv

DESCRIPTION : Déplace ou renomme des fichiers et répertoires
 POURQUOI : Renommage et déplacement sans copie – l'original disparaît de la source.
 CONTEXTE : Réorganisation, renommage en masse, déplacement de données.
 OPTIONS :

```

-i          Demande confirmation avant écrasement
-u          Déplace seulement si source plus récente
-v          Affiche les opérations effectuées
-n          Ne pas écraser les fichiers existants
--backup    Sauvegarde les fichiers qui seraient écrasés

```

EXEMPLES :

```

$ mv ancien.txt nouveau.txt
$ mv *.log /var/log/archive/
$ mv -iv /tmp/test /home/user/

```

[USER] rm

DESCRIPTION : Supprime des fichiers ou répertoires – IRRÉVERSIBLE
 POURQUOI : Suppression définitive (pas de corbeille en terminal). Commande puissante – une erreur peut détruire le système entier.
 CONTEXTE : Nettoyage de fichiers temporaires, suppression de logs anciens.
 OPTIONS :

```

-r / -R     Suppression récursive – nécessaire pour les dossiers
-f          Force la suppression sans confirmation
-i          Demande confirmation pour chaque fichier
-I          Une seule confirmation si plus de 3 fichiers
-v          Affiche chaque fichier supprimé

```

EXEMPLES :

```

$ rm fichier.txt
$ rm -rf /tmp/dossier/
$ rm -i *.tmp

```

⚠ DANGER : Ne JAMAIS exécuter : rm -rf / ou rm -rf /*

[USER] mkdir

DESCRIPTION : Crée un ou plusieurs répertoires
 POURQUOI : Structurer l'arborescence de travail, initialiser des projets.
 CONTEXTE : Création de projets, déploiement, scripts d'installation.
 OPTIONS :

```

-p          Crée les répertoires parents manquants sans erreur
-m MODE     Définit les permissions à la création (ex: 755)
-v          Affiche les répertoires créés

```

EXEMPLES :

```

$ mkdir monDossier
$ mkdir -p /opt/app/config/ssl
$ mkdir -m 700 ~/.ssh
$ mkdir -pv /var/www/html/assets/{css,js,img}

```

[USER] rmdir

DESCRIPTION : Supprime des répertoires VIDES uniquement
 POURQUOI : Plus sûr que rm -r car refuse de supprimer un dossier non vide.
 CONTEXTE : Nettoyage de structures vides après déplacement de fichiers.
 OPTIONS :

```

-p          Supprime le répertoire et ses parents s'ils deviennent vides
-v          Affiche les répertoires supprimés

```

```
EXEMPLES      :  
$ rmdir dossier_vide  
$ rmdir -p a/b/c
```

```
[USER] find
```

DESCRIPTION : Recherche de fichiers et répertoires selon des critères très précis
POURQUOI : Outil de recherche le plus puissant de Linux. Retrouve tout : par nom, taille, date, permissions, type, propriétaire... et peut exécuter des commandes sur les résultats.
CONTEXTE : Audit sécurité, nettoyage, recherche de fichiers de config, scripts.

OPTIONS :

-name "motif"	Recherche par nom (sensible à la casse)
-iname "motif"	Recherche par nom (insensible à la casse)
-type f	Fichiers normaux uniquement
-type d	Répertoires uniquement
-type l	Liens symboliques uniquement
-size +10M	Fichiers de plus de 10 Mo
-size -1k	Fichiers de moins de 1 Ko
-mtime -7	Modifiés dans les 7 derniers jours
-mtime +30	Modifiés il y a plus de 30 jours
-user nom	Appartient à cet utilisateur
-group nom	Appartient à ce groupe
-perm 644	A exactement ces permissions
-perm /4000	A le bit SUID activé (risque sécurité)
-exec CMD {} \;	Exécute CMD sur chaque résultat
-exec CMD {} +	Exécute CMD sur tous les résultats d'un coup
-delete	Supprime les fichiers trouvés
-maxdepth N	Limite la profondeur de recherche à N niveaux
-mindepth N	Ignore les N premiers niveaux
-not / !	Inverse le critère suivant
-o	OU logique entre deux critères

EXEMPLES :

```
$ find /var/log -name "*.log" -mtime +30 -delete  
$ find / -perm /4000 -type f 2>/dev/null  
$ find /home -size +100M -type f  
$ find . -name "*.conf" -exec grep -l "password" {} \;
```

```
[USER] ln
```

DESCRIPTION : Crée des liens entre fichiers (physiques ou symboliques)
POURQUOI : Les liens symboliques permettent d'avoir plusieurs chemins vers le même fichier – très utilisé pour la gestion de versions d'applications.
CONTEXTE : Gestion de versions, raccourcis système, partage de fichiers.

OPTIONS :

-s	Crée un lien symbolique (soft link) – le plus courant
-f	Supprime le lien existant avant de créer le nouveau
-n	Ne pas déréférencer le lien destination
-v	Affiche les liens créés
-r	Crée des liens symboliques relatifs

EXEMPLES :

```
$ ln -s /usr/bin/python3 /usr/local/bin/python  
$ ln -sf /etc/nginx/sites-available/mon-site /etc/nginx/sites-enabled/  
$ ln fichier.txt lien_physique.txt
```

```
[USER] touch
```

DESCRIPTION : Crée un fichier vide ou met à jour les timestamps d'un fichier existant
POURQUOI : Création rapide de fichiers vides, mise à jour de dates pour déclencher des processus basés sur les timestamps.
CONTEXTE : Scripts, création de fichiers de verrou (lock files), tests.

OPTIONS :

-a	Met à jour uniquement la date d'accès
-m	Met à jour uniquement la date de modification
-t STAMP	Définit une date précise (format: YYYYMMDDhhmm)
-r FICHIER	Copie les timestamps d'un autre fichier

EXEMPLES :

```
$ touch nouveau_fichier.txt  
$ touch -t 202301011200 fichier.txt  
$ touch /tmp/lock.pid
```

[USER] stat

DESCRIPTION : Affiche les métadonnées détaillées d'un fichier ou système de fichiers
POURQUOI : Obtenir toutes les informations sur un fichier : inode, permissions en octal, timestamps précis (accès, modification, changement), taille réelle.
CONTEXTE : Audit, débogage de permissions, scripts comparant des dates de fichiers.
OPTIONS :
-f Affiche des infos sur le système de fichiers
-c FORMAT Affiche selon un format personnalisé
-L Suit les liens symboliques
--printf=FORMAT Affiche avec un format de printf
EXEMPLES :
\$ stat /etc/passwd
\$ stat -f /var
\$ stat -c "%n %a %U" /etc/shadow

[USER] file

DESCRIPTION : Détermine le type réel d'un fichier (indépendamment de l'extension)
POURQUOI : Un fichier .txt peut être un exécutable, un .jpg peut être du texte. file lit la signature magique (magic bytes) du fichier.
CONTEXTE : Sécurité, analyse de fichiers inconnus, forensics, scripting.
OPTIONS :
-b Affiche le type sans le nom du fichier
-i Affiche le type MIME
-z Inspecte le contenu des fichiers compressés
-L Suit les liens symboliques
-s Lit aussi les fichiers spéciaux (block/char devices)
EXEMPLES :
\$ file /bin/bash
\$ file -i image.png
\$ file fichier_inconnu

[USER] locate / plocate

DESCRIPTION : Recherche rapide de fichiers via une base de données pré-indexée
POURQUOI : Beaucoup plus rapide que find car utilise une base indexée. Inconvénient : la base peut être obsolète (mise à jour par updatedb).
CONTEXTE : Recherche rapide de fichiers dont on connaît une partie du nom.
OPTIONS :
-i Insensible à la casse
-c Affiche seulement le nombre de résultats
-l N Limite à N résultats
-r REGEX Utilise une expression régulière
-e Vérifie que les fichiers existent encore
EXEMPLES :
\$ locate nginx.conf
\$ locate -i "*.PDF"
\$ locate -c passwd

[SUDO] updatedb

DESCRIPTION : Met à jour la base de données de locate
POURQUOI : Sans mise à jour régulière, locate retourne des résultats obsolètes. Généralement exécuté automatiquement par cron chaque nuit.
CONTEXTE : Administration système, après installation de nouveaux logiciels.
OPTIONS :
--output=FICHIER Spécifie le fichier de base de données
--prunefts=TYPES Exclut certains types de systèmes de fichiers
--prunepaths=CHEMINS Exclut certains chemins de l'indexation
EXEMPLES :
\$ sudo updatedb
\$ sudo updatedb --prunepaths="/proc /sys /tmp"

[USER] tree

DESCRIPTION : Affiche l'arborescence d'un répertoire sous forme d'arbre visuel
POURQUOI : Visualiser la structure d'un projet ou d'un répertoire de manière claire et hiérarchique.
CONTEXTE : Documentation de projet, exploration de structure, revue de code.
OPTIONS :

```

-a          Inclut les fichiers cachés
-d          Affiche uniquement les répertoires
-L N        Limite la profondeur à N niveaux
-f          Affiche les chemins complets
-h          Tailles lisibles
-I "motif"  Exclut les fichiers correspondant au motif
--dirsfirst Affiche les dossiers avant les fichiers
-o FICHIER  Écrit la sortie dans un fichier

```

EXEMPLES :

```

$ tree /etc/nginx
$ tree -L 2 /var/www
$ tree -d -L 3 /usr

```

[USER] basename

DESCRIPTION : Extrait le nom de fichier d'un chemin complet

POURQUOI : Dans les scripts, récupérer uniquement le nom du fichier sans le chemin.

CONTEXTE : Scripts shell de traitement de fichiers.

OPTIONS :

```

  SUFFIX          Supprime le suffixe spécifié du résultat
  -a              Traite chaque argument séparément
  -s SUFFIX       Supprime le suffixe spécifié

```

EXEMPLES :

```

$ basename /home/user/document.pdf
$ basename /home/user/document.pdf .pdf
$ basename -s .txt /tmp/fichier.txt

```

[USER] dirname

DESCRIPTION : Extrait le répertoire parent d'un chemin complet

POURQUOI : Complémentaire de basename – obtenir le dossier contenant un fichier.

CONTEXTE : Scripts shell, construction de chemins dynamiques.

EXEMPLES :

```

$ dirname /home/user/document.pdf
$ SCRIPT_DIR=$(dirname "$0")

```

[USER] realpath

DESCRIPTION : Résout et affiche le chemin absolu canonique d'un fichier

POURQUOI : Résout les liens symboliques, les ../ et ./ pour obtenir le vrai chemin.

CONTEXTE : Scripts nécessitant des chemins absolus fiables.

OPTIONS :

```

  -m          Ne vérifie pas l'existence du fichier
  -s          Ne résout pas les liens symboliques
  --relative-to=DIR  Chemin relatif à un répertoire donné

```

EXEMPLES :

```

$ realpath ../config.txt
$ realpath -m /tmp/../etc/./passwd

```

[USER] readlink

DESCRIPTION : Affiche la cible d'un lien symbolique

POURQUOI : Savoir vers quoi pointe un lien symbolique.

CONTEXTE : Débogage de liens, scripts de vérification.

OPTIONS :

```

  -f          Résout récursivement tous les liens jusqu'au fichier réel
  -e          Comme -f mais vérifie que la cible existe
  -m          Comme -f sans vérification d'existence
  -n          Supprime le saut de ligne final

```

EXEMPLES :

```

$ readlink /usr/bin/python
$ readlink -f /etc/alternatives/java

```

[USER] namei

DESCRIPTION : Affiche le chemin d'accès composant par composant avec les types

POURQUOI : Déboguer les problèmes de permissions sur un chemin : voit exactement où le droit manque dans la hiérarchie.

CONTEXTE : Diagnostic de permissions, sécurité, administration.

OPTIONS :

```
-l          Format long avec permissions
-m          Affiche les modes (permissions)
-o          Affiche les propriétaires
-x          Affiche les permissions de montage
```

```
EXEMPLES    :
$ namei -l /var/www/html/index.html
$ namei -mo /home/user/.ssh/authorized_keys
```

```
[USER] dir
```

```
DESCRIPTION : Similaire à ls, liste le contenu d'un répertoire
POURQUOI    : Alternative à ls avec comportement légèrement différent par défaut
              (pas de coloration automatique, format colonnes).
CONTEXTE     : Scripts nécessitant un comportement prévisible indépendamment des alias.
EXEMPLES     :
$ dir /etc
$ dir -l /tmp
```

```
[USER] vdir
```

```
DESCRIPTION : Comme dir mais avec le format long par défaut (équivalent à ls -l)
POURQUOI    : Listing détaillé par défaut sans avoir à spécifier -l.
CONTEXTE     : Scripts, utilisation rapide quand on veut toujours le format long.
EXEMPLES     :
$ vdir /home/user
```

```
[USER] install
```

```
DESCRIPTION : Copie des fichiers en définissant permissions et propriétaire en une fois
POURQUOI    : Utilisé dans les Makefiles et scripts d'installation pour déployer des
              fichiers avec les bonnes permissions directement.
CONTEXTE     : Makefile, scripts d'installation, déploiement de packages.
OPTIONS      :
-m MODE      Définit les permissions (ex: 755)
-o PROPRIÉTAIRE Définit le propriétaire
-g GROUPE    Définit le groupe
-d           Crée des répertoires au lieu de copier des fichiers
-v           Mode verbeux
-b           Crée une sauvegarde avant écrasement
EXEMPLES     :
$ install -m 755 monscript.sh /usr/local/bin/
$ install -d -m 700 /etc/monapp/
```

```
[USER] shred
```

```
DESCRIPTION : Supprime un fichier de manière sécurisée en écrasant son contenu
POURQUOI    : Un simple rm ne supprime pas réellement les données – elles restent
              récupérables. shred écrase avec des données aléatoires avant suppression.
CONTEXTE     : Suppression sécurisée de fichiers sensibles (mots de passe, clés privées).
OPTIONS      :
-n N         Nombre de passes d'écrasement (défaut: 3)
-u          Supprime le fichier après écrasement
-v          Mode verbeux
-z          Ajoute une passe de zéros pour masquer l'écrasement
-f          Change les permissions si nécessaire pour écrire
EXEMPLES     :
$ shred -uvz -n 5 fichier_secret.key
$ shred -u mot_de_passe.txt
```

```
[USER] truncate
```

```
DESCRIPTION : Redimensionne un fichier à une taille donnée (vide ou étend)
POURQUOI    : Vider rapidement un fichier de log sans le supprimer (conserve les
              descripteurs de fichiers ouverts), ou créer des fichiers de taille fixe.
CONTEXTE     : Gestion de logs, création de fichiers tests, images disque.
OPTIONS      :
-s TAILLE    Définit la taille (ex: 0, 1G, 100M)
-r FICHIER_REF Utilise la taille d'un fichier de référence
EXEMPLES     :
$ truncate -s 0 /var/log/apache2/access.log
```

```
$ truncate -s 1G fichier_test.img
$ truncate -s 100M disque_virtuel.raw
```

```
[USER] mktmp
```

DESCRIPTION : Crée un fichier ou répertoire temporaire avec un nom unique
POURQUOI : Évite les conflits de noms dans les scripts et prévient les attaques par lien symbolique sur /tmp (symlink attacks).
CONTEXTE : Scripts shell sécurisés nécessitant des fichiers temporaires.
OPTIONS :

- d Crée un répertoire temporaire
- p RÉPERTOIRE Crée dans ce répertoire au lieu de /tmp
- t MODÈLE Spécifie un modèle de nom
- suffix=SUFFIXE Ajoute un suffixe au nom

EXEMPLES :

```
$ TMP=$(mktmp)
$ TMPDIR=$(mktmp -d)
$ mktmp /tmp/monscript.XXXXXX
```

```
[USER] symlinks
```

DESCRIPTION : Analyse et gère les liens symboliques dans un répertoire
POURQUOI : Trouver les liens cassés (dangling symlinks), les liens relatifs/absolus, et les nettoyer.
CONTEXTE : Maintenance système, nettoyage après désinstallation de packages.
OPTIONS :

- r Analyse récursive
- d Supprime les liens cassés
- v Affiche tous les liens (pas seulement les cassés)
- c Convertit les liens absolus en liens relatifs
- t Mode test (n'effectue pas les modifications)

EXEMPLES :

```
$ symlinks -r /usr/local
$ symlinks -rd /etc
```

```
[USER] hardlink
```

DESCRIPTION : Consolide les fichiers identiques en liens physiques pour économiser l'espace
POURQUOI : Si plusieurs fichiers ont exactement le même contenu, hardlink les fusionne en un seul inode – économie d'espace disque significative.
CONTEXTE : Optimisation d'espace disque sur les serveurs de sauvegarde.
OPTIONS :

- v Mode verbeux
- n Mode simulation (dry-run)
- f Force même entre différents systèmes de fichiers

EXEMPLES :

```
$ hardlink -v /var/cache/pacman/pkg
$ hardlink -n /backup/
```

```
[USER] chattr
```

DESCRIPTION : Modifie les attributs étendus des fichiers ext2/ext3/ext4
POURQUOI : Protéger des fichiers critiques même contre root : un fichier avec l'attribut 'i' (immutable) ne peut pas être modifié même par root.
CONTEXTE : Sécurisation de fichiers critiques (/etc/passwd, configs importantes).
OPTIONS :

- +i Rend le fichier immuable (immutable)
- i Retire l'attribut immuable
- +a Append only : seuls les ajouts sont autorisés (logs)
- a Retire l'attribut append-only
- +d Exclut du dump (sauvegarde)
- R Applique récursivement
- v VERSION Définit le numéro de version du fichier

EXEMPLES :

```
$ sudo chattr +i /etc/resolv.conf
$ sudo chattr +a /var/log/secure.log
$ sudo chattr -i /etc/resolv.conf
```

```
[USER] lsattr
```

DESCRIPTION : Affiche les attributs étendus des fichiers (complément de chatter)
 POURQUOI : Voir quels attributs spéciaux (immutable, append-only...) sont appliqués.
 CONTEXTE : Diagnostic de fichiers qui résistent aux modifications même en root.
 OPTIONS :
 -R Récursif
 -a Affiche aussi les fichiers cachés
 -d Affiche les attributs du répertoire lui-même
 -v Affiche les numéros de version
 EXEMPLES :
 \$ lsattr /etc/resolv.conf
 \$ lsattr -Ra /etc/

[USER] attr

DESCRIPTION : Manipule les attributs étendus (xattr) des systèmes de fichiers
 POURQUOI : Les xattrs permettent de stocker des métadonnées arbitraires sur les fichiers (utilisés par SELinux, ACL, tags...)
 CONTEXTE : SELinux, gestion de métadonnées, compatibilité Samba.
 OPTIONS :
 -g NOM Lit la valeur d'un attribut étendu
 -s NOM VALEUR Définit un attribut étendu
 -r NOM Supprime un attribut étendu
 -l Liste tous les attributs étendus
 EXEMPLES :
 \$ attr -l /etc/passwd
 \$ attr -g selinux /etc/passwd
 \$ attr -s commentaire "important" fichier.txt

[USER] getfattr / setfattr

DESCRIPTION : Lecture (getfattr) et écriture (setfattr) des attributs étendus POSIX
 POURQUOI : Interface standard POSIX pour manipuler les xattrs – utilisé par Samba, SELinux, systèmes de fichiers avancés.
 CONTEXTE : Administration SELinux, compatibilité Samba, métadonnées avancées.
 OPTIONS getfattr:
 -n NOM Récupère un attribut spécifique
 -d Affiche tous les attributs
 -R Récursif
 OPTIONS setfattr:
 -n NOM Nom de l'attribut à définir
 -v VALEUR Valeur de l'attribut
 -x NOM Supprime l'attribut
 EXEMPLES :
 \$ getfattr -d /etc/passwd
 \$ setfattr -n user.commentaire -v "fichier important" /home/user/doc.txt

[USER] compsize

DESCRIPTION : Affiche la taille réelle des fichiers compressés sur btrfs ou EROFS
 POURQUOI : Sur un système de fichiers avec compression (btrfs), la taille affichée par ls est la taille logique, pas la taille compressée réelle sur disque.
 CONTEXTE : Administration btrfs, audit d'espace disque sur systèmes compressés.
 EXEMPLES :
 \$ compsize /var/lib
 \$ compsize -x /home

[USER] dircolors

DESCRIPTION : Configure les couleurs utilisées par ls pour les différents types de fichiers
 POURQUOI : Personnaliser les couleurs d'affichage de ls selon les extensions ou types.
 CONTEXTE : Personnalisation du shell, scripts de configuration d'environnement.
 OPTIONS :
 -b Génère la config pour bash/sh
 -c Génère la config pour csh/tcsh
 -p Affiche la configuration par défaut
 EXEMPLES :
 \$ eval \$(dircolors -b)
 \$ dircolors -p > ~/.dircolors

[USER] pathchk


```
DESCRIPTION : Vérifie la validité et la portabilité d'un chemin de fichier
POURQUOI : S'assurer qu'un chemin est valide sur différents systèmes POSIX avant
           de l'utiliser dans un script portable.
CONTEXTE : Scripts portables, validation d'entrées utilisateur.
OPTIONS :
    -p Vérifie la portabilité POSIX stricte
    --portability Comme -p
EXEMPLES :
    $ pathchk /home/user/mon fichier.txt
    $ pathchk -p /tmp/test
```

[USER] link

```
DESCRIPTION : Crée un lien physique entre deux fichiers (appel système direct)
POURQUOI : Version bas niveau de ln sans options – utile pour les scripts qui
           veulent un comportement strictement prévisible.
CONTEXTE : Scripts système, utilisation bas niveau.
EXEMPLES :
    $ link fichier.txt lien.txt
```

[USER] unlink

```
DESCRIPTION : Supprime un lien vers un fichier (appel système direct)
POURQUOI : Version bas niveau de rm pour supprimer exactement un fichier/liens.
CONTEXTE : Scripts système, suppression précise d'un lien.
EXEMPLES :
    $ unlink lien_symbolique
    $ unlink fichier.txt
```

[USER] rename

```
DESCRIPTION : Renomme des fichiers en masse selon une expression régulière
POURQUOI : mv ne permet pas le renommage en masse avec regex. rename permet
           de changer l'extension de 1000 fichiers en une commande.
CONTEXTE : Renommage en masse, changement d'extension, normalisation de noms.
OPTIONS :
    -v Affiche chaque renommage effectué
    -n Mode simulation (dry-run)
    -f Force l'écrasement
EXEMPLES :
    $ rename 's/.txt/.md/' *.txt
    $ rename -v 'y/A-Z/a-z/' *.TXT
    $ rename 's/espace/ /_/' *
```

[USER] dos2unix / unix2dos / mac2unix / unix2mac

```
DESCRIPTION : Convertit les fins de lignes entre formats DOS/Windows (CRLF),
              Unix/Linux (LF) et Mac Classic (CR)
POURQUOI : Les fichiers créés sous Windows ont des fins de ligne CRLF (\r\n)
           qui causent des bugs dans les scripts shell Linux.
CONTEXTE : Développement cross-platform, correction de scripts venant de Windows.
OPTIONS :
    -v Mode verbeux
    -n ENTREE SORTIE Crée un nouveau fichier au lieu de modifier sur place
    -k Préserve la date de modification
EXEMPLES :
    $ dos2unix script.sh
    $ unix2dos rapport.txt
    $ dos2unix -v *.sh
```